

DecentralizedVoting con Privacidad ZK

Sistema de votación en Ethereum con Zero-Knowledge Proofs

Irina Lamboglia — Legajo 18090/3

Facultad de Informática — Universidad Nacional de La Plata | 03/08/2026

Repositorio: github.com/IrinaLamboglia/DecentralizedVoting

Contrato Sepolia: 0xe5d603bef9800084c43a24c3ad66c56668d1577c

1. Abstract

Este trabajo presenta un sistema de votación descentralizado sobre Ethereum que intenta resolver dos problemas que están en tensión: que la votación sea transparente y auditable, pero que al mismo tiempo los votos sean privados. La primera parte es el contrato DecentralizedVoting, deployado en la testnet Sepolia, que garantiza que cada wallet vote exactamente una vez y que el resultado sea verificable por cualquier persona. La segunda parte estudia e implementa Semaphore, un protocolo de Zero-Knowledge Proofs que permite que un votante demuestre que tiene derecho a votar sin revelar quién es. El código completo, los scripts y todas las capturas de pantalla están disponibles en el repositorio del proyecto.

2. El problema

Las votaciones tradicionales tienen problemas conocidos: no son auditables en tiempo real, dependen de autoridades centrales y son difíciles de verificar para el ciudadano común. Blockchain parece una solución natural porque todo queda registrado públicamente.

El problema que no es tan obvio es que en Ethereum todo es público, incluyendo qué wallet votó por quién. En una elección real eso viola el principio básico del voto secreto y puede generar presión sobre los votantes. El desafío es construir un sistema que sea:

- Transparente: cualquiera puede verificar que el proceso fue correcto y que los votos se contaron bien.
- Anónimo: nadie puede saber qué votó cada persona específica, ni siquiera el administrador.
- Resistente al doble voto: cada votante habilitado puede emitir su voto una sola vez.

3. Solución base: DecentralizedVoting

3.1 Diseño del contrato

El contrato está escrito en Solidity 0.8.18. La estructura central es simple: un array de candidatos con su conteo de votos, y un mapping que registra si cada dirección ya votó.

```
struct Candidate { string name; uint256 voteCount; }
mapping(address => bool) public hasVoted;
```

El owner del contrato es quien despliega la elección y tiene control para abrirla y cerrarla con startVoting() y endVoting(). Cualquier wallet puede llamar a vote(uint256 index) exactamente una vez — si intenta votar de nuevo el contrato lo rechaza con el mensaje 'Ya votaste en esta eleccion'. Al cerrar, getWinner() recorre los candidatos y devuelve el ganador. El contrato emite tres eventos (Voted, VotingStarted, VotingEnded) que la interfaz web escucha para actualizarse en tiempo real sin necesidad de recargar la página.

3.2 Deploy y prueba en Sepolia

El contrato fue desplegado el 28 de julio de 2026 con tres candidatos: Roberto, Pepe y Juan, bajo el nombre 'Eleccion UNLP 2026'. A continuación se muestra el deploy confirmado en Etherscan y la tabla con el historial completo de transacciones:

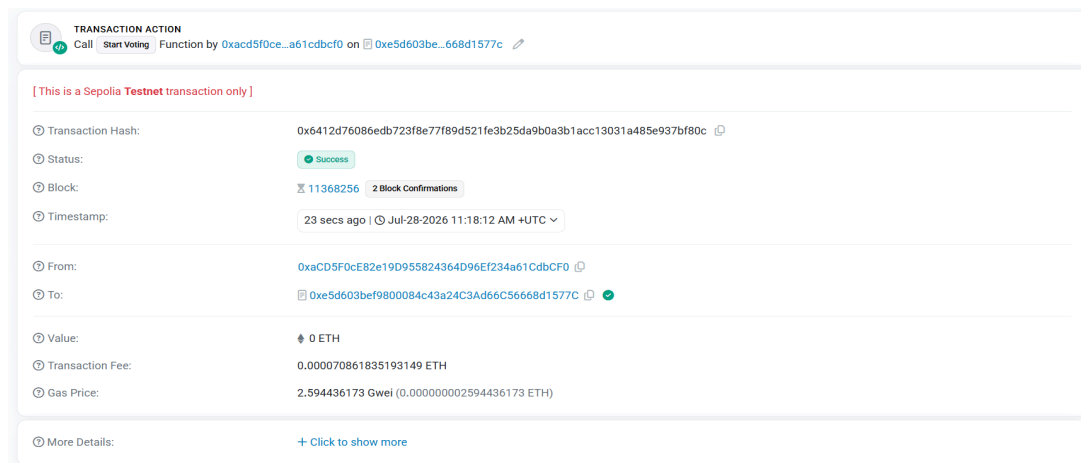


Figura 1: Deploy del contrato en Sepolia — Status: Success, bloque 11368233

Método	TX Hash	Bloque	Estado	Gas Fee
Deploy	0x2879bd90...99bcb	11368233	✓ Success	0.00280 ETH
startVoting	0x6412d760...f80c	11368256	✓ Success	0.000070 ETH
vote(o)	0x9435b362...aa2a	11368266	✓ Success	0.000185 ETH
endVoting	0xa9907483...6885	11368283	✓ Success	0.000069 ETH

→ Ver figuras 2, 3, 11, 12 en github.com/IrinaLamboglia/DecentralizedVoting/tree/main/imagenes/contrato%20y%20deploy

3.3 Interfaz de usuario

Se desarrolló una interfaz web en HTML y JavaScript puro usando ethers.js v5 y MetaMask como proveedor de red. No depende de ningún framework — todo corre en el browser. Al conectar MetaMask, la interfaz detecta automáticamente si la cuenta ya votó y oculta o muestra los botones correspondientes. El panel owner aparece solo para la wallet que deployó el contrato. Los resultados se actualizan en tiempo real escuchando los eventos on-chain.

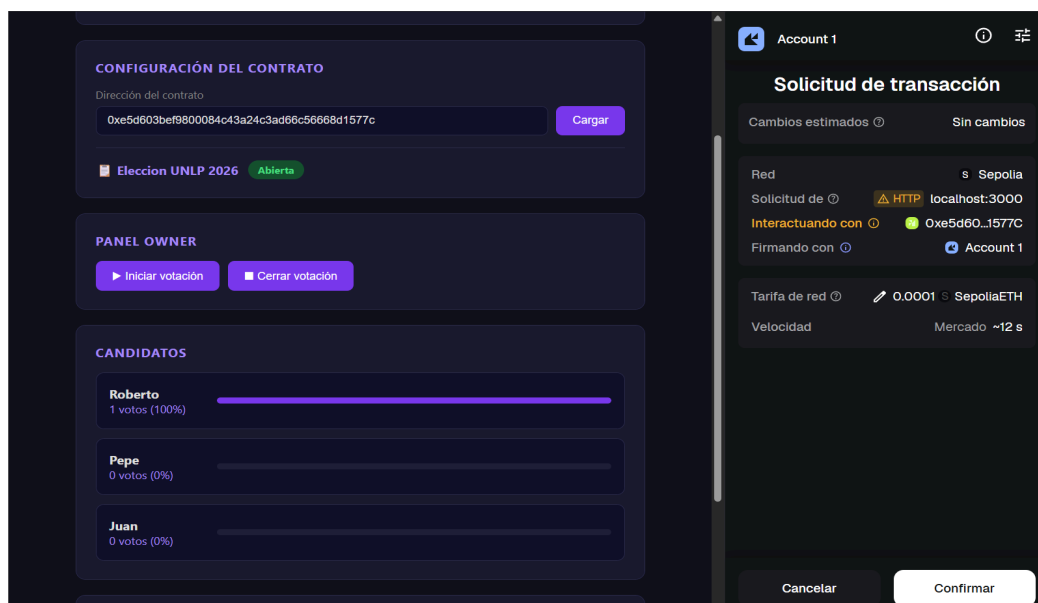


Figura 2: Interfaz web con los candidatos Roberto, Pepe y Juan y botones de votación activos — Account conectada en Sepolia

→ Ver figuras 4, 5, 6, 7 en github.com/IrinaLamboglia/DecentralizedVoting/tree/main/imagenes/interfaz

4. Extensión con Zero-Knowledge Proofs

4.1 El problema de privacidad del contrato base

El contrato base resuelve la transparencia pero no la privacidad. El mapping `hasVoted` vincula directamente una dirección Ethereum con el acto de haber votado. Cualquier persona que conozca la dirección de alguien puede saber si esa persona participó. Para resolver esto se estudió el concepto de Zero-Knowledge Proofs visto en la clase de Privacidad y Criptomonedas.

Un ZK-SNARK permite demostrar que se conoce algo sin revelar qué es. En el contexto de una votación, un votante puede demostrar que pertenece al padrón de votantes habilitados sin revelar cuál de ellos es. Esto conecta directamente con el concepto de 'Pertenencia a un grupo' visto en clase: la prueba demuestra que tu identidad está en un conjunto sin decir cuál.

4.2 Semaphore

Semaphore es un protocolo de ZK-SNARKs diseñado específicamente para votación anónima. En lugar de usar direcciones Ethereum visibles, cada votante genera una identidad criptográfica privada compuesta por una clave privada EdDSA que nunca sale de su dispositivo, y un commitment público que es el hash Poseidon de esa clave. El commitment es lo único que se comparte.

Los commitments de todos los votantes habilitados se organizan en un árbol de Merkle. El contrato on-chain solo necesita guardar el root de ese árbol — no las identidades individuales. Para votar, el votante genera una prueba ZK que demuestra tres cosas a la vez: que su commitment está en el árbol (es un votante válido), que conoce la clave privada correspondiente (es realmente él), y que no votó antes en esta elección (el nullifier es único por scope). El contrato puede verificar todo esto sin saber en ningún momento quién es el votante.

4.3 Implementación

Se desarrollaron dos implementaciones del protocolo ZK:

La primera es un script en Node.js (`demo.js`) que usa el SDK oficial de Semaphore v4.14.2 y ejecuta el flujo completo: genera identidades ZK reales con criptografía EdDSA, arma el árbol de Merkle, genera una prueba ZK-SNARK real descargando los circuitos de la Trusted Setup Ceremony oficial (~14MB), verifica la prueba exitosamente, y demuestra que un segundo intento de voto genera el mismo nullifier y sería rechazado. La captura a continuación muestra el output real del script:

```
Run 'npm audit' for details.
npm notice
npm notice New minor version of npm available! 11.16.0 -> 11.18.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.18.0
npm notice To update run: npm install -g npm@11.18.0
npm notice

C:\Users\irina\semaphore-demo>node demo.js
=== DEMO: Votación anónima con Semaphore (ZK-SNARKs) ===

[1] Generando identidades de votantes...
Alice commitment: 18310021228928004845135876424747485528005645195610072703208864586566066099205
Bob commitment: 4183333204265380269668045523331701809819792798681783466348776285871186773045
Carla commitment: 8036355992739931203539116660402275484079825972018261766622624567365230689863
(La clave privada de cada uno NUNCA se comparte)

[2] Creando grupo de votantes habilitados (Merkle Tree)...
Root del árbol: 21010272889525666452029227811667099229419618672056567669502213292973705473954
Cantidad de miembros: 3

[3] Bob genera una prueba ZK de que es un votante válido...
Prueba generada ✓
Nullifier : 17704040342234815147896623564771293190034643465880443672462942597250187425043
Merkle root: 21010272889525666452029227811667099229419618672056567669502213292973705473954
Mensaje (voto): 1
-> Noten que NO aparece el commitment de Bob en ningún lado.
Solo se sabe que ALGUIEN del grupo votó.

[4] Verificando la prueba (sin conocer la identidad)...
¿Prueba válida? ✓ SÍ

[5] Intento de DOBLE VOTO (mismo votante, mismo scope)...
Nullifier repetido: SÍ -> el contrato lo rechazaría

=== Conclusión ===
El contrato on-chain solo necesita guardar:
- El root del árbol de votantes habilitados
- Los nullifiers ya usados para este 'scope'
Nunca necesita saber qué wallet corresponde a cada voto.
```

Figura 3: Output real de `node demo.js` — identidades ZK generadas, árbol de Merkle, prueba ZK verificada y detección de doble voto

La segunda implementación integra la generación de identidades ZK directamente en la interfaz web. Al conectar MetaMask, la interfaz genera automáticamente una identidad ZK para el votante (clave privada EdDSA + commitment Poseidon) que se muestra en un badge amarillo. Al hacer click en Votar, el sistema intenta generar la prueba ZK antes de enviar la transacción on-chain, mostrando en el log los pasos intermedios: identidad generada, árbol de Merkle armado, e intento de prueba. La generación completa de la prueba ZK en el browser requiere una versión compilada del SDK con `snarkjs` bundleado, lo que excede el alcance de este trabajo y queda como extensión futura.

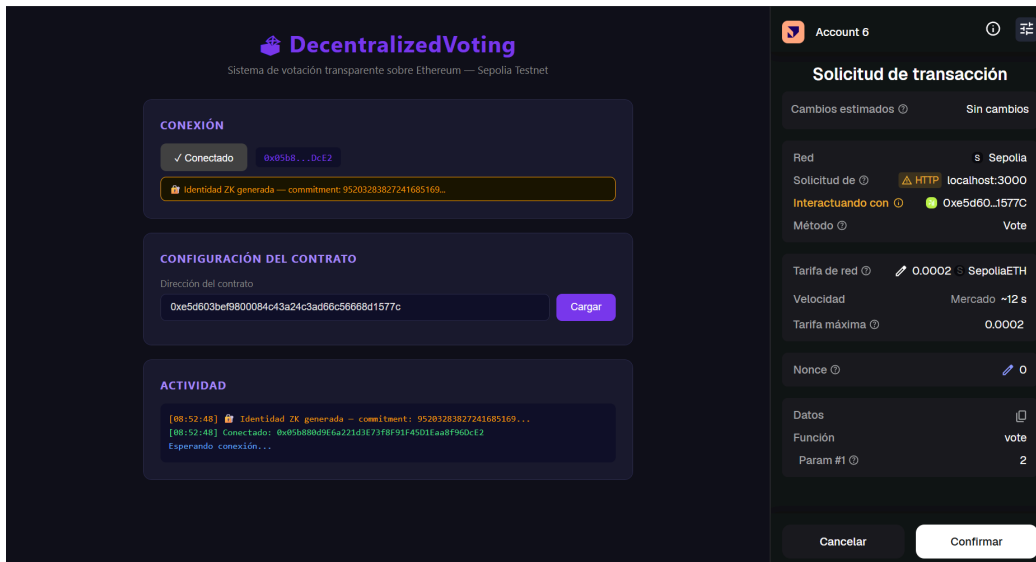


Figura 4: Identidad ZK generada automáticamente en el browser al conectar MetaMask — commitment visible en el badge amarillo

→ Ver figuras 8, 9, 10, 13 en github.com/IrinaLamboglia/DecentralizedVoting/tree/main/imagenes/ZK

5. Conclusiones

El trabajo demuestra que es posible construir un sistema de votación que resuelve la tensión entre transparencia y privacidad usando herramientas disponibles en el ecosistema Ethereum. El contrato base garantiza un proceso auditable y resistente al doble voto, con transacciones reales verificables en Sepolia. La interfaz web permite a cualquier usuario interactuar con el contrato usando MetaMask sin conocimientos técnicos.

El estudio e implementación de Semaphore mostró que la privacidad en votaciones blockchain es un problema resuelto a nivel de protocolo. Los ZK-SNARKs permiten garantizar anonimato total manteniendo la verificabilidad del resultado — exactamente lo que se necesita para una elección justa. La generación de identidades ZK en el browser funciona correctamente; la integración completa del generador de pruebas on-chain queda como trabajo futuro junto con el deploy del verificador Semaphore en Sepolia y la evaluación de costos en redes L2.

Referencias

Repositorio del proyecto con código y capturas: github.com/IrinaLamboglia/DecentralizedVoting

Semaphore Protocol — github.com/semaphore-protocol/semaphore

Groth, J. (2016). On the Size of Pairing-based Non-interactive Arguments. EUROCRYPT 2016.

OpenZeppelin Contracts v4.8.3 — github.com/OpenZeppelin/openzeppelin-contracts

Contrato en Etherscan Sepolia — sepolia.etherscan.io/address/0xe5d603bef9800084c43a24c3ad66c56668d1577c